

Logic and Proof

Mathematics for Computing – Application-Based Questions

Arya T S

M.Tech Artificial Intelligence & Software Engineering
Cochin University of Science and Technology (CUSAT)

October 2026

Introduction

Applying fundamental concepts of mathematical reasoning to computing scenarios

Software Requirement Verification

A software system has the following requirements:

- Every registered user must have a unique user ID.
- Every user ID is associated with exactly one account.
- An inactive account must still have a unique user ID.

A developer claims:

“If two accounts have the same user ID, they must belong to the same user.”

Introduction – Question 1 (Continued)

Question

Determine whether the developer's statement logically follows from the given requirements.

Explain your reasoning and identify the mathematical idea involved.

Let

$$P = \text{"two accounts have the same user ID."}$$

and let

$$Q = \text{"the accounts belong to the same user."}$$

Express the developer's statement in logical form.

Introduction – Question 1: Answer

Solution

The requirements state that every user ID is unique.

Therefore, two different accounts cannot have the same user ID.

Hence,

$$P = \text{False.}$$

The developer's statement has the form

$$P \rightarrow Q.$$

Introduction – Question 1: Answer (Continued)

Solution

Since the antecedent P is false under the given requirements,

$$P \rightarrow Q$$

is logically true.

This illustrates the concept of **vacuous truth** in logical implication.

Conclusion:

The implication is formally true because its antecedent describes an impossible situation. However, the requirements do not independently establish the claimed relationship between the accounts.

AI Classification Rule

An AI-based spam filter uses the following rule:

A message is spam if it contains a suspicious link or more than five promotional keywords.

Consider a message containing a suspicious link but only two promotional keywords.
Let

$P =$ “The message contains a suspicious link.”

Introduction – Question 2 (Continued)

Question

Let

$Q =$ “The message contains more than five promotional keywords.”

Determine whether the message is classified as spam.

Express the spam rule using P and Q .

Why is knowing only the truth value of Q insufficient to determine whether the message is spam?

Explain your answer using the logical operator involved.

Introduction – Question 2: Answer

Solution

The message contains a suspicious link.

Therefore,

$$P = \text{True}.$$

The spam rule uses the OR operator:

$$P \vee Q$$

The message contains only two promotional keywords, so it does not contain more than five.

Therefore,

$$Q = \text{False}.$$

Introduction – Question 2: Answer (Continued)

Solution

Substituting the truth values,

$$P \vee Q = \text{True} \vee \text{False} = \text{True}.$$

Therefore, the message is classified as spam.

Knowing only Q is insufficient because the result of

$$P \vee Q$$

depends on both propositions.

Key idea:

A disjunction is true when at least one of its propositions is true.

Introduction – Question 3

Database Access Control

A database administrator defines the following access policy:

A user can access a confidential file if the user is an administrator or has been explicitly granted permission.

Let

A = “the user is an administrator”

and

P = “the user has explicit permission.”

Introduction – Question 3 (Continued)

Question

A user who is not an administrator and has not been granted permission attempts to access the file.

Determine whether access should be permitted.

Represent the access policy using a logical expression.

Explain how the truth values of A and P determine the result.

Introduction – Question 3: Answer

Solution

The policy states that access is allowed when either condition is satisfied.
Therefore, the logical expression is

$$A \vee P.$$

The user is not an administrator, so

$$A = \text{False}.$$

The user has not been granted permission, so

$$P = \text{False}.$$

Introduction – Question 3: Answer (Continued)

Solution

Substituting the truth values,

$$A \vee P = \text{False} \vee \text{False} = \text{False}.$$

Therefore,

Access is denied.

Key idea:

For a disjunction $A \vee P$ to be true, at least one of A or P must be true.

Machine Learning Model Evaluation

A machine-learning model is designed to flag fraudulent transactions.

The system uses the statement:

“If a transaction is flagged as suspicious, then it is fraudulent.”

During testing, a transaction is flagged as suspicious, but is later found to be legitimate.

Introduction – Question 4 (Continued)

Question

Let

$P =$ “the transaction is flagged as suspicious”

and

$Q =$ “the transaction is fraudulent.”

Determine whether the observation invalidates the stated implication.
Explain why or why not using the logical meaning of implication.
Give a condition under which the implication would definitely be false.

Introduction – Question 4: Answer

Solution

The statement is represented as

$$P \rightarrow Q.$$

In the observed case, the transaction was flagged as suspicious.
Therefore,

$$P = \text{True}.$$

However, the transaction was later found to be legitimate.
Therefore,

$$Q = \text{False}.$$

Introduction – Question 4: Answer (Continued)

Solution

An implication is false only in the case

$$\text{True} \rightarrow \text{False}.$$

Here,

$$P \rightarrow Q = \text{True} \rightarrow \text{False} = \text{False}.$$

Therefore, the observation invalidates the stated implication.

This is an example of a **false positive**: the system flags a legitimate transaction as suspicious.

Introduction – Question 5

Autonomous System Decision

An autonomous vehicle uses the following rule:

If the road is wet and visibility is poor, the vehicle reduces its speed.

Let

W = “the road is wet”,

V = “visibility is poor”,

S = “the vehicle reduces speed.”

Introduction – Question 5 (Continued)

Question

The following observations are recorded:

Condition	Observation
Road is wet	True
Visibility is poor	False
Vehicle reduces speed	False

Represent the decision rule using W , V , and S .

Determine whether the observed situation violates the decision rule.

Introduction – Question 5: Answer

Solution

The rule states:

$$(W \wedge V) \rightarrow S$$

From the observations,

$$W = \text{True},$$

$$V = \text{False},$$

$$S = \text{False}.$$

Therefore,

Introduction – Question 5: Answer (Continued)

Solution

The complete implication becomes

$$\text{False} \rightarrow \text{False}.$$

An implication with a false antecedent is true.

Therefore, the observed situation does not violate the decision rule.

A violating observation would be

$$W = \text{True}, \quad V = \text{True}, \quad S = \text{False}.$$

Introduction – Question 5: Answer (Final)

Solution

For the violating observation,

$$(W \wedge V) \rightarrow S = \text{True} \rightarrow \text{False}.$$

Therefore,

$$(W \wedge V) \rightarrow S = \text{False}.$$

Conclusion:

The rule is violated when both required conditions occur, but the required action does not occur.

Proofs

Constructing, evaluating, and communicating mathematical proofs

Algorithm Correctness

A programmer claims that the following algorithm correctly determines whether an integer n is even:

If $n \bmod 2 = 0$, return “Even”;

otherwise, return “Odd”.

The programmer wants to justify that the algorithm is correct for every integer input.

Proofs – Question 1 (Continued)

Question

Construct a mathematical proof that the algorithm correctly classifies every integer as either even or odd.

Your proof should explain why every integer n must fall into one of the two cases represented by the algorithm.

Proofs – Question 1: Answer

Proof

Let n be any integer.

By the division algorithm, when n is divided by 2, the remainder must be either 0 or 1. Therefore,

$$n = 2q$$

for some integer q , or

$$n = 2q + 1$$

for some integer q .

Proofs – Question 1: Answer (Continued)

Proof

If

$$n = 2q,$$

then

$$n \bmod 2 = 0,$$

so the algorithm returns “Even”.

If

$$n = 2q + 1,$$

then

$$n \bmod 2 = 1,$$

so the algorithm returns “Odd”.

Data Validation Rule

A data-processing system stores integer values that are required to be multiples of 3.
A developer claims:

“The sum of two valid stored values is also a valid stored value.”

Suppose a and b are two valid stored values.

Proofs – Question 2 (Continued)

Question

Prove the developer's claim mathematically.

Your proof should begin by expressing a and b as multiples of 3 and show that their sum is also a multiple of 3.

Proofs – Question 2: Answer

Proof

Since a and b are valid stored values, both are multiples of 3.
Therefore, there exist integers m and n such that

$$a = 3m$$

and

$$b = 3n.$$

Consider their sum:

$$a + b = 3m + 3n.$$

Proofs – Question 2: Answer (Continued)

Proof

Factor out 3:

$$a + b = 3(m + n).$$

Since $m + n$ is an integer,

$$a + b$$

is a multiple of 3.

Therefore, the sum of two valid stored values is also valid.

$$a + b \text{ is a multiple of } 3$$

Hence, the developer's claim is proved.

Network Identifier Property

A network system assigns integer identifiers to devices.

A particular subsystem uses only identifiers of the form

$$2k + 1,$$

where k is an integer.

An engineer observes that combining two such identifiers by addition always produces an even number.

Proofs – Question 3 (Continued)

Question

Prove that the sum of any two identifiers of the form $2k + 1$ is even.

Let the two identifiers be

$$a = 2m + 1$$

and

$$b = 2n + 1,$$

where $m, n \in \mathbb{Z}$.

Show that $a + b$ can be written in the form $2r$ for some integer r .

Proofs – Question 3: Answer

Proof

Let

$$a = 2m + 1$$

and

$$b = 2n + 1,$$

where

$$m, n \in \mathbb{Z}.$$

Their sum is

$$a + b = (2m + 1) + (2n + 1).$$

Simplifying,

$$a + b = 2m + 2n + 2.$$

Proofs – Question 3: Answer (Continued)

Proof

Factor out 2:

$$a + b = 2(m + n + 1).$$

Since m , n , and 1 are integers,

$$m + n + 1 \in \mathbb{Z}.$$

Let

$$r = m + n + 1.$$

Then

$$a + b = 2r.$$

Therefore, $a + b$ is even.

The sum of two odd integers is even.

Proofs – Question 4

Software Access Rule

A software system assigns each user an integer access score.

The security team claims:

“If a user’s access score is divisible by 4, then it is even.”

A developer attempts to prove this by saying:

“All even numbers are divisible by 4, so any number divisible by 4 is even.”

Proofs – Question 4 (Continued)

Question

Determine whether the developer's reasoning is a valid proof.

If it is not valid, identify the error in the reasoning and provide a correct proof of the security team's claim.

Proofs – Question 4: Answer

Analysis

The developer's reasoning is not valid.

The statement

“All even numbers are divisible by 4”

is false.

For example,

2

is even, but 2 is not divisible by 4.

Therefore, the starting statement used by the developer is incorrect.

Proofs – Question 4: Answer (Continued)

Correct Proof

Let n be an integer divisible by 4.

Then there exists an integer k such that

$$n = 4k.$$

Since

$$4k = 2(2k),$$

we can write

$$n = 2(2k).$$

Because $2k$ is an integer, n is even.

Therefore,

$$\boxed{4 \mid n \Rightarrow 2 \mid n}.$$

Hence, the security team's claim is true.

Hashing System Property

A hashing system maps an integer input n to

$$h(n) = n^2 + n.$$

A developer claims that the hash value $h(n)$ is always even, regardless of the integer input.

For example,

$$h(3) = 3^2 + 3 = 12.$$

Proofs – Question 5 (Continued)

Question

Prove that

$$n^2 + n$$

is even for every integer n .

Your proof should consider the possible parity of n and show that the result is even in every case.

Proofs – Question 5: Answer

Proof

Let n be any integer.

Every integer is either even or odd.

Case 1: n is even

Then there exists an integer k such that

$$n = 2k.$$

Therefore,

$$n^2 + n = (2k)^2 + 2k = 4k^2 + 2k.$$

Proofs – Question 5: Answer (Continued)

Proof

Factor out 2:

$$n^2 + n = 2(2k^2 + k).$$

Therefore, $n^2 + n$ is even.

Case 2: n is odd

Let

$$n = 2k + 1.$$

Then

$$\begin{aligned} n^2 + n &= (2k + 1)^2 + (2k + 1) \\ &= 4k^2 + 6k + 2 \\ &= 2(2k^2 + 3k + 1). \end{aligned}$$

Therefore, $n^2 + n$ is even.

Proofs – Question 5: Answer (Final)

Conclusion

In both possible cases,

$$n^2 + n$$

is even.

Therefore,

$$n^2 + n \text{ is even for every } n \in \mathbb{Z}.$$

Hence, the developer's claim about the hashing function is correct.

Proof idea:

The argument uses a proof by cases based on whether the integer is even or odd.

Propositions

Representing and evaluating statements in computing contexts

Propositions – Question 1

Question

A software system checks whether a user can access a protected database. The following conditions are defined:

- P : The user has entered a valid password.
- Q : The user has completed two-factor authentication.

The system grants access only when both conditions are satisfied.

- 1 Express the access rule as a proposition.
- 2 Determine whether the user is granted access if P is true and Q is false.

Propositions – Question 1: Answer

Step 1: Represent the Rule

The system grants access only when both conditions are satisfied.
Therefore, the access rule is

$$P \wedge Q.$$

Here,

P = valid password,

and

Q = two-factor authentication completed.

Propositions – Question 1: Answer (Continued)

Step 2: Evaluate the Rule

The given values are

$$P = \text{True}, \quad Q = \text{False}.$$

Therefore,

$$P \wedge Q = \text{True} \wedge \text{False} = \text{False}.$$

Propositions – Question 1: Answer (Conclusion)

Conclusion

Since the access condition is false, the user is not granted access.

Access Denied

Propositions – Question 2

Question

An intrusion detection system raises an alert when either a suspicious login location or an unusual login time is detected.

Let

P : suspicious login location is detected,

Q : unusual login time is detected.

The system records that the login occurred from a suspicious location, but at a normal time.

Propositions – Question 2 (Continued)

Question

- 1 Express the alert condition as a proposition.
- 2 Determine whether an alert is raised.

Propositions – Question 2: Answer

Step 1: Represent the Rule

The system raises an alert when either condition occurs.
Therefore, the alert condition is

$$P \vee Q.$$

Propositions – Question 2: Answer (Continued)

Step 2: Evaluate the Conditions

The recorded values are

$$P = \text{True}, \quad Q = \text{False}.$$

Thus,

$$P \vee Q = \text{True} \vee \text{False} = \text{True}.$$

Propositions – Question 2: Answer (Conclusion)

Conclusion

Since the alert condition is true, the intrusion detection system raises an alert.

Alert Raised

Propositions – Question 3

Question

A machine-learning service accepts a model for deployment only if the model has completed testing and has an accuracy of at least 90

Let

P : the model has completed testing,

Q : the model has accuracy at least 90%.

A model has completed testing, but its accuracy is only 87

Propositions – Question 3 (Continued)

Question

Determine whether the model can be deployed and justify your answer using a compound proposition.

Propositions – Question 3: Answer

Step 1: Identify the Conditions

Both conditions are required for deployment.
Therefore, the deployment condition is

$$P \wedge Q.$$

For the given model,

$$P = \text{True}.$$

Propositions – Question 3: Answer (Continued)

Step 2: Evaluate Accuracy

The model has an accuracy of 87

Since

$$87\% < 90\%,$$

the accuracy condition is false:

$$Q = \text{False}.$$

Propositions – Question 3: Answer (Continued)

Step 3: Evaluate the Proposition

Using

$$P = \text{True}, \quad Q = \text{False},$$

we obtain

$$P \wedge Q = \text{True} \wedge \text{False} = \text{False}.$$

Therefore, the deployment condition is false.

Propositions – Question 3: Answer (Conclusion)

Conclusion

The model does not satisfy both conditions required for deployment.
Hence,

The model cannot be deployed.

The compound proposition

$$P \wedge Q$$

represents a rule in which all required conditions must be satisfied.

Propositions – Question 4

Question

A cloud server is considered operational when it is either in the *Running* state or in the *Standby* state.

Let

P : server is in the Running state,

Q : server is in the Standby state.

A monitoring system reports that the server is neither running nor in standby mode.

Propositions – Question 4 (Continued)

Question

- 1 Represent the operational condition as a proposition.
- 2 Determine whether the server is operational.
- 3 State the negation of the operational condition.

Propositions – Question 4: Answer

Step 1: Represent the Condition

The server is operational when it is in either state.

Therefore,

$$P \vee Q.$$

The monitoring system reports

$$P = \text{False}, \quad Q = \text{False}.$$

Propositions – Question 4: Answer (Continued)

Step 2: Determine the State

Using the given values,

$$P \vee Q = \text{False} \vee \text{False} = \text{False}.$$

Therefore, the server does not satisfy the operational condition.

Propositions – Question 4: Answer (Continued)

Step 3: Negate the Condition

The operational condition is

$$P \vee Q.$$

Its negation is

$$\neg(P \vee Q).$$

Using De Morgan's law,

$$\neg(P \vee Q) \equiv \neg P \wedge \neg Q.$$

Propositions – Question 4: Answer (Conclusion)

Interpretation

The expression

$$\neg P \wedge \neg Q$$

means that the server is neither in the Running state nor in the Standby state.

Therefore,

$$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$$

is the required negation.

Propositions – Question 5

Question

A data-processing pipeline performs a backup when the following rule is satisfied:

Backup occurs if the dataset is modified and the storage system is available.

Let

P : dataset is modified,

Q : storage system is available,

R : backup occurs.

Propositions – Question 5 (Continued)

Question

The system records that the dataset was modified and the storage system was unavailable.

- 1 Express the backup rule as a proposition.
- 2 Determine whether the backup should occur.

Propositions – Question 5: Answer

Step 1: Identify the Conditions

The backup rule contains three propositions:

P : dataset is modified,

Q : storage system is available,

R : backup occurs.

Propositions – Question 5: Answer (Continued)

Step 2: Represent the Rule

The dataset must be modified and the storage system must be available.
Therefore, the required condition is

$$P \wedge Q.$$

If this condition is satisfied, a backup occurs.
Thus,

$$(P \wedge Q) \rightarrow R.$$

Propositions – Question 5: Answer (Continued)

Step 3: Use the Recorded Values

The dataset was modified.

Therefore,

$$P = \text{True}.$$

The storage system was unavailable.

Therefore,

$$Q = \text{False}.$$

Propositions – Question 5: Answer (Continued)

Step 4: Evaluate the Antecedent

The antecedent is

$$P \wedge Q.$$

Substituting the recorded values,

$$P \wedge Q = \text{True} \wedge \text{False} = \text{False}.$$

Therefore,

$$P \wedge Q = \text{False}.$$

Propositions – Question 5: Answer (Continued)

Step 5: Evaluate the Implication

The complete proposition is

$$(P \wedge Q) \rightarrow R.$$

Since

$$P \wedge Q = \text{False},$$

the proposition becomes

$$\text{False} \rightarrow R.$$

An implication with a false antecedent is true.

Therefore,

Propositions – Question 5: Answer (Conclusion)

Interpretation

The truth of the implication does not mean that a backup actually occurred.
The required condition

$$P \wedge Q$$

was false because the storage system was unavailable.
Therefore, the backup rule was not violated.

Propositions – Question 5: Answer (Conclusion)

Final Result

The system cannot require a backup when the storage system is unavailable.
Hence,

The backup rule is satisfied, but no backup is required.

Predicates and Quantifiers

Predicates and Quantifiers – Question 1

Question

A university maintains a database of students enrolled in a programming course.
Let

$P(x)$: x has submitted the programming assignment.

Consider the statement:

Every student has submitted the programming assignment.

Predicates and Quantifiers – Question 1 (Continued)

Question

- 1 Express the statement using a universal quantifier.
- 2 State what the predicate $P(x)$ represents.

Predicates and Quantifiers – Question 1: Answer

Step 1: Identify the Predicate

The predicate is

$P(x)$: x has submitted the programming assignment.

Here, x represents a student in the course.

Predicates and Quantifiers – Question 1: Answer (Continued)

Step 2: Interpret the Predicate

The predicate becomes either true or false depending on whether a particular student has submitted the assignment.

The statement says that this property is true for every student.

Therefore, we use the universal quantifier.

Predicates and Quantifiers – Question 1: Answer (Conclusion)

Step 3: Apply the Universal Quantifier

The universal quantifier is

$$\forall x.$$

Therefore,

$$\boxed{\forall x P(x)}$$

where x ranges over all students enrolled in the course.

Predicates and Quantifiers – Question 2

Question

A cloud service monitors a collection of servers.

Let

$P(x)$: server x is operational.

The administrator wants to verify whether at least one server is currently operational.

Predicates and Quantifiers – Question 2 (Continued)

Question

- 1 Express this requirement using an existential quantifier.
- 2 Explain what the resulting statement means.

Predicates and Quantifiers – Question 2: Answer

Step 1: Identify the Requirement

The requirement is that **at least one** server is operational.

The phrase “at least one” indicates the use of the existential quantifier.

Predicates and Quantifiers – Question 2: Answer (Continued)

Step 2: Apply the Existential Quantifier

The predicate is

$P(x)$: server x is operational.

The existential quantifier is

$\exists x.$

Therefore,

$\boxed{\exists x P(x)}.$

Predicates and Quantifiers – Question 2: Answer (Conclusion)

Interpretation

The statement

$$\exists x P(x)$$

means that there exists at least one server x for which $P(x)$ is true.

Hence,

At least one server is operational.

Predicates and Quantifiers – Question 3

Question

A security system checks every login attempt.

Let

$P(x)$: login attempt x is from an authorized user,

$Q(x)$: login attempt x is successful.

The security policy states:

Every successful login is from an authorized user.

Predicates and Quantifiers – Question 3 (Continued)

Question

Express this policy using predicates, logical implication, and a universal quantifier.

Predicates and Quantifiers – Question 3: Answer

Step 1: Translate the Policy

The statement says that a successful login must be from an authorized user.
Therefore, for any login attempt x ,

$$Q(x) \rightarrow P(x).$$

Predicates and Quantifiers – Question 3: Answer (Continued)

Step 2: Apply the Universal Quantifier

The policy applies to **every** login attempt.

Therefore, we use

$$\forall x.$$

Combining the quantifier with the implication gives

$$\boxed{\forall x [Q(x) \rightarrow P(x)]}.$$

Predicates and Quantifiers – Question 3: Answer (Conclusion)

Interpretation

The expression

$$\forall x [Q(x) \rightarrow P(x)]$$

means that for every login attempt, if the login is successful, then the user must be authorized.

Predicates and Quantifiers – Question 4

Question

A machine-learning dataset contains a collection of records.

Let

$P(x)$: record x contains missing values,

$Q(x)$: record x is removed during preprocessing.

The preprocessing rule is:

Every record containing missing values is removed.

Predicates and Quantifiers – Question 4 (Continued)

Question

Express this rule using predicates, implication, and a universal quantifier.

Predicates and Quantifiers – Question 4: Answer

Step 1: Identify the Logical Relationship

The rule states that whenever a record contains missing values, that record is removed. Therefore, for any record x ,

$$P(x) \rightarrow Q(x).$$

Predicates and Quantifiers – Question 4: Answer (Continued)

Step 2: Apply the Universal Quantifier

The preprocessing rule applies to every record in the dataset.
Therefore, we use

$$\forall x.$$

Combining the quantifier with the implication gives

$$\boxed{\forall x [P(x) \rightarrow Q(x)]}.$$

Predicates and Quantifiers – Question 4: Answer (Conclusion)

Interpretation

The expression

$$\forall x [P(x) \rightarrow Q(x)]$$

means that for every record x , if x contains missing values, then x is removed during preprocessing.

Predicates and Quantifiers – Question 5

Question

A software testing system examines a set of test cases.

Let

$P(x)$: test case x exposes a software defect.

The testing report states:

No test case exposes a software defect.

Predicates and Quantifiers – Question 5 (Continued)

Question

Express this statement using a universal quantifier and predicate negation.

Predicates and Quantifiers – Question 5: Answer

Step 1: Interpret the Statement

The statement says that for every test case, the predicate $P(x)$ is false. Therefore, we can express the statement as

$$\forall x \neg P(x).$$

Predicates and Quantifiers – Question 5: Answer (Continued)

Step 2: Interpret the Expression

The expression

$$\forall x \neg P(x)$$

means that every test case fails to satisfy the predicate $P(x)$.
In other words, no test case exposes a software defect.

Predicates and Quantifiers – Question 5: Answer (Continued)

Step 3: Equivalent Form

The statement can also be expressed by saying that there does not exist a test case for which $P(x)$ is true.

Therefore,

$$\forall x \neg P(x) \equiv \neg \exists x P(x)$$

Predicates and Quantifiers – Question 5: Answer (Conclusion)

Final Interpretation

Both expressions represent the same statement:

$$\forall x \neg P(x)$$

and

$$\neg \exists x P(x).$$

Hence,

No test case exposes a software defect.

Truth Tables

Question 1

A software system grants access only when a user enters a valid password and successfully completes two-factor authentication.

Let:

P = Valid password

Q = Two-factor authentication successful

Represent the access condition using a logical expression and construct its truth table. Identify the conditions under which access is granted.

Question 1 – Answer

The system requires both conditions to be satisfied.
Therefore, the logical expression is:

$$P \wedge Q$$

There are four possible combinations of truth values for P and Q .

Question 1 – Answer

The truth table is:

P	Q	$P \wedge Q$
T	T	T
T	F	F
F	T	F
F	F	F

The expression is true only when both P and Q are true.

Question 1 – Answer

Therefore, access is granted only when:

$$P = T, \quad Q = T$$

This means:

The password is valid and two-factor authentication is successful.

For every other combination, access is denied.

Question 2

A backup system raises an alert when either the primary storage device fails or the backup storage device fails.

Let:

P = Primary storage fails

Q = Backup storage fails

Represent the alert condition using a logical expression and construct its truth table.
Determine when the alert is not raised.

Question 2 – Answer

The alert is raised when at least one of the two storage devices fails.
Therefore, the logical expression is:

$$P \vee Q$$

There are four possible combinations of truth values for P and Q .

Question 2 – Answer

The truth table is:

P	Q	$P \vee Q$
T	T	T
T	F	T
F	T	T
F	F	F

The expression is false only when both P and Q are false.

Question 2 – Answer

Therefore, the alert is not raised only when:

$$P = F, \quad Q = F$$

This means:

Neither the primary nor the backup storage device has failed.

Question 3

An intrusion detection system follows the rule:

If suspicious activity is detected, the security team must be notified.

Let:

P = Suspicious activity is detected

Q = Security team is notified

Represent the rule using an implication and construct its truth table. Identify the condition that represents a violation of the rule.

Question 3 – Answer

The rule has the form:

If P , then Q .

Therefore, the logical expression is:

$$P \rightarrow Q$$

Question 3 – Answer

The truth table is:

P	Q	$P \rightarrow Q$
T	T	T
T	F	F
F	T	T
F	F	T

An implication is false only when the antecedent is true and the consequent is false.

Question 3 – Answer

The rule is violated when:

$$P = T, \quad Q = F$$

Therefore:

Suspicious activity is detected, but the security team is not notified.

This is the only combination that makes the implication false.

Question 4

A cloud service considers a server healthy when both its CPU usage is within the permitted range and its memory usage is within the permitted range. However, an emergency override can also declare the server healthy.

Question 4

Let:

P = CPU usage is within the permitted range

Q = Memory usage is within the permitted range

R = Emergency override is active

The health condition is:

$$(P \wedge Q) \vee R$$

Construct the complete truth table and determine how many combinations result in the server being considered healthy.

Question 4 – Answer

The logical expression is:

$$(P \wedge Q) \vee R$$

There are three propositions.

Therefore, the number of possible combinations is:

$$2^3 = 8$$

So, the complete truth table contains eight rows.

Question 4 – Answer

The truth table is:

P	Q	R	$P \wedge Q$	$(P \wedge Q) \vee R$
T	T	T	T	T
T	T	F	T	T
T	F	T	F	T
T	F	F	F	F
F	T	T	F	T
F	T	F	F	F
F	F	T	F	T
F	F	F	F	F

Question 4 – Answer

The final column is true in four cases.
Therefore:

4 combinations

result in the server being considered healthy.

Question 4 – Answer

The server is considered healthy when either:

- both CPU and memory conditions are satisfied, or
- the emergency override is active.

The truth table examines every possible combination of the three conditions.

Question 5

An autonomous vehicle follows a safety policy:

If an obstacle is detected, then the vehicle must either brake or change direction.

Let:

P = Obstacle is detected

Q = Vehicle brakes

R = Vehicle changes direction

Represent the policy using a logical expression and construct its complete truth table.
Identify all situations in which the safety policy is violated.

Question 5 – Answer

The vehicle must brake or change direction whenever an obstacle is detected.
Therefore, the consequent is:

$$Q \vee R$$

The complete policy is:

$$P \rightarrow (Q \vee R)$$

Question 5 – Answer

Since there are three propositions, the truth table contains:

$$2^3 = 8$$

possible combinations.

We first evaluate:

$$Q \vee R$$

Question 5 – Answer

The complete truth table is:

P	Q	R	$Q \vee R$	$P \rightarrow (Q \vee R)$
T	T	T	T	T
T	T	F	T	T
T	F	T	T	T
T	F	F	F	F
F	T	T	T	T
F	T	F	T	T
F	F	T	T	T
F	F	F	F	T

Question 5 – Answer

The policy is violated only when:

$$P = T, \quad Q = F, \quad R = F$$

In this situation:

An obstacle is detected, but the vehicle neither brakes nor changes direction.

Question 5 – Answer

Therefore, the safety policy is violated in exactly one of the eight possible situations.

$$P = T, Q = F, R = F$$

This shows how a truth table can systematically identify unsafe combinations of system conditions.

First Order Logic

Question 1

A university maintains a database of students enrolled in a programming course.
Let:

$P(x)$ = x has completed the programming course

Express the following requirement using first order logic:

Every student enrolled in the course has completed the programming course.

Question 1 – Answer

The statement applies to every student in the course.
Therefore, we use the universal quantifier:

$$\forall x$$

The predicate is:

$$P(x) = x \text{ has completed the programming course}$$

Question 1 – Answer

The required statement is:

$$\boxed{\forall x P(x)}$$

This means that for every student x in the relevant domain, x has completed the programming course.

Question 2

A cloud computing system contains several servers. The system requirement states:

At least one server must be operational before a service can be provided.

Let:

$P(x)$ = x is an operational server

Express this requirement using first order logic.

Question 2 – Answer

The phrase *at least one* indicates that we need an existential quantifier.
The existential quantifier is:

$$\exists x$$

The predicate is:

$$P(x) = x \text{ is an operational server}$$

Question 2 – Answer

Therefore, the requirement can be written as:

$$\boxed{\exists x P(x)}$$

This states that there exists at least one server that is operational.

Question 3

A company's authentication system records user login attempts.

Let:

$P(x) = x$ is a successful login

$Q(x) = x$ is associated with an authorized user

The security requirement states:

Every successful login must be associated with an authorized user.

Express the requirement using first order logic.

Question 3 – Answer

The statement applies to every login attempt.
Therefore, we use:

$$\forall x$$

The condition that must hold for every successful login is:

$$P(x) \rightarrow Q(x)$$

Question 3 – Answer

Combining the universal quantifier with the implication gives:

$$\boxed{\forall x [P(x) \rightarrow Q(x)]}$$

This means that whenever a login is successful, it must be associated with an authorized user.

Question 4

An artificial intelligence system processes a collection of training records.
Let:

$P(x)$ = x contains missing values

$Q(x)$ = x is removed from the training dataset

The preprocessing requirement states:

Every record containing missing values is removed from the training dataset.

Express the requirement using first order logic.

Question 4 – Answer

The statement applies to every training record.

Therefore, we use:

$$\forall x$$

If a record contains missing values, it must be removed.

This gives the implication:

$$P(x) \rightarrow Q(x)$$

Question 4 – Answer

Therefore, the complete first order logic expression is:

$$\boxed{\forall x [P(x) \rightarrow Q(x)]}$$

The expression states that every record satisfying $P(x)$ must also satisfy $Q(x)$.

Question 5

A software testing system contains a collection of test cases.

Let:

$P(x) = x$ exposes a software defect

The testing report claims:

No test case exposes a software defect.

Express this statement using first order logic and give an equivalent form using an existential quantifier.

Question 5 – Answer

The statement says that every test case does not expose a defect.
Therefore, we can use the universal quantifier:

$$\forall x$$

The statement becomes:

$$\boxed{\forall x \neg P(x)}$$

Question 5 – Answer

The statement can also be expressed by saying that there does not exist a test case that exposes a defect.

Using the existential quantifier:

$$\neg \exists x P(x)$$

Question 5 – Answer

Therefore, the two equivalent forms are:

$$\forall x \neg P(x)$$

and

$$\neg \exists x P(x)$$

Hence:

$$\boxed{\forall x \neg P(x) \equiv \neg \exists x P(x)}$$

Both statements mean that no test case exposes a software defect.

Satisfiability

Question 1

A software system has the following access rules:

- A user can access the system if the password is valid.
- A user cannot access the system if the password is invalid.

Let:

P = Password is valid

Q = User can access the system

Determine whether the following set of conditions is satisfiable:

$$P \rightarrow Q$$

$$\neg P \rightarrow \neg Q$$

Question 1 – Answer

Consider the possible values of P .

If:

$$P = T$$

then the first condition requires:

$$Q = T$$

If:

$$P = F$$

then the second condition requires:

$$Q = F$$

Question 1 – Answer

A satisfying assignment is therefore:

$$P = T, \quad Q = T$$

Both conditions are true:

$$T \rightarrow T = T$$

$$F \rightarrow F = T$$

Therefore:

The set of conditions is satisfiable.

Question 2

A database system has the following requirements:

- A transaction must be approved before execution.
- The transaction is executed.
- The transaction is not approved.

Let:

P = Transaction is approved

Q = Transaction is executed

Determine whether the following conditions are satisfiable:

$$P \rightarrow Q$$

$$Q$$

$$\neg P$$

Question 2 – Answer

The conditions require:

$$Q = T$$

and:

$$P = F$$

Substituting these values into the implication gives:

$$F \rightarrow T = T$$

Question 2 – Answer

Therefore, the assignment

$$P = F, \quad Q = T$$

satisfies all three conditions.

Thus:

The conditions are satisfiable.

The implication does not require P to be true whenever Q is true.

Question 3

An authentication system has the following rules:

- If a user is authenticated, access is granted.
- Access is granted.
- The user is not authenticated.

Let:

P = User is authenticated

Q = Access is granted

Determine whether the following set of statements is satisfiable:

$$P \rightarrow Q$$

$$Q$$

$$\neg P$$

Question 3 – Answer

The conditions require:

$$Q = T$$

and:

$$P = F$$

The implication becomes:

$$F \rightarrow T = T$$

Question 3 – Answer

Therefore:

$$P = F, \quad Q = T$$

makes all the statements true.

Hence:

The statements are satisfiable.

This example shows that an implication alone does not allow us to infer that its antecedent is true.

Question 4

A cloud service has the following requirements:

- A server is operational if and only if it is connected to the network.
- The server is operational.
- The server is not connected to the network.

Let:

P = Server is operational

Q = Server is connected to the network

Determine whether the requirements are satisfiable.

Question 4 – Answer

The first requirement is:

$$P \leftrightarrow Q$$

The other requirements state:

$$P = T$$

and:

$$Q = F$$

Question 4 – Answer

Substituting these values into the biconditional gives:

$$T \leftrightarrow F = F$$

Therefore, the first requirement is violated.

There is no assignment that can satisfy all three requirements.

Question 4 – Answer

Hence:

The requirements are unsatisfiable.

The contradiction occurs because the biconditional requires P and Q to have the same truth value.

Question 5

A machine-learning deployment system uses the following policy:

- A model is deployed only if it passes testing.
- A model is deployed only if its accuracy is at least 90%.
- The model is deployed.
- The model does not pass testing.

Question 5

Let:

P = Model passes testing

Q = Model accuracy is at least 90%

R = Model is deployed

Determine whether the following set of conditions is satisfiable:

$$P \rightarrow R$$

$$Q \rightarrow R$$

$$R$$

$$\neg P$$

Question 5 – Answer

The given conditions require:

$$R = T$$

and:

$$P = F$$

Consider the first implication:

$$P \rightarrow R$$

Substituting the values gives:

$$F \rightarrow T = T$$

Question 5 – Answer

The second implication is:

$$Q \rightarrow R$$

Since $R = T$, this implication is true regardless of the value of Q .

For example:

$$Q = T$$

gives:

$$T \rightarrow T = T$$

Question 5 – Answer

A satisfying assignment is:

$$P = F, \quad Q = T, \quad R = T$$

All four conditions are satisfied.

Therefore:

The set of conditions is satisfiable.

Pattern of Proof

Question 1

A program generates an even number whenever its integer input is doubled.
The output is given by:

$$f(n) = 2n$$

where n is an integer.

Using a direct proof, show that the output of the program is always even.

Question 1 – Answer

Let n be any integer.

By the definition of an even integer, a number is even if it can be written in the form:

$$2k$$

for some integer k .

Question 1 – Answer

The program produces:

$$f(n) = 2n$$

Since n is an integer, $2n$ has the required form $2k$, where:

$$k = n$$

Therefore:

$$f(n) \text{ is even}$$

for every integer n .

Question 2

A data validation system accepts an integer identifier only when it is divisible by 6.
A developer claims that every accepted identifier must also be divisible by 3.
Use a direct proof to verify the developer's claim.

Question 2 – Answer

Let n be an accepted identifier.

Since the system accepts n only when it is divisible by 6, there exists an integer k such that:

$$n = 6k$$

Question 2 – Answer

Rewrite $6k$ as:

$$n = 3(2k)$$

Since k is an integer, $2k$ is also an integer.

Therefore, n is divisible by 3.

Hence:

$$6 \mid n \Rightarrow 3 \mid n$$

The developer's claim is proved.

Question 3

A computer system assigns two integer values a and b to two processes. Suppose the system guarantees that a and b have the same parity.

A monitoring program claims that $a + b$ must be even.

Prove the claim by considering the possible parity of a and b .

Question 3 – Answer

Since a and b have the same parity, there are two possible cases:

- both are even,
- both are odd.

Question 3 – Answer

Case 1: Both are even

Let:

$$a = 2m, \quad b = 2n$$

Then:

$$a + b = 2m + 2n$$

$$a + b = 2(m + n)$$

Therefore, $a + b$ is even.

Question 3 – Answer

Case 2: Both are odd

Let:

$$a = 2m + 1, \quad b = 2n + 1$$

Then:

$$a + b = 2m + 1 + 2n + 1$$

$$a + b = 2(m + n + 1)$$

Therefore, $a + b$ is even.

Since both possible cases give an even result:

$a + b$ is always even

Question 4

A software system stores a positive integer n and computes:

$$n(n + 1)$$

A developer claims that the result is always even.

Identify an appropriate pattern of proof and use it to establish the claim.

Question 4 – Answer

The appropriate proof pattern is a **proof by cases**.
For any integer n , exactly one of the following is true:

$$n = 2k$$

or

$$n = 2k + 1$$

Question 4 – Answer

Case 1: $n = 2k$

Then:

$$n(n + 1) = 2k(2k + 1)$$

Therefore:

$$n(n + 1) = 2[k(2k + 1)]$$

So the result is even.

Question 4 – Answer

Case 2: $n = 2k + 1$

Then:

$$n(n + 1) = (2k + 1)(2k + 2)$$

Therefore:

$$n(n + 1) = 2[(2k + 1)(k + 1)]$$

So the result is also even.

Hence:

$n(n + 1)$ is always even

Question 5

A distributed computing system contains n processing tasks. The system requires at least one communication link between every pair of tasks.

The number of required links is:

$$L = \frac{n(n-1)}{2}$$

Show that the number of links is always an integer.

Use an appropriate proof pattern based on the parity of n .

Question 5 – Answer

We need to show that:

$$\frac{n(n-1)}{2}$$

is an integer for every positive integer n .

We use a proof by cases based on whether n is even or odd.

Question 5 – Answer

Case 1: n is even

Let:

$$n = 2k$$

Then:

$$\frac{n(n-1)}{2} = \frac{2k(2k-1)}{2}$$

Therefore:

$$L = k(2k-1)$$

which is an integer.

Question 5 – Answer

Case 2: n is odd

Let:

$$n = 2k + 1$$

Then:

$$\frac{n(n-1)}{2} = \frac{(2k+1)(2k)}{2}$$

Therefore:

$$L = k(2k+1)$$

which is an integer.

Question 5 – Answer

In both possible cases, the number of links is an integer.
Therefore:

$$\frac{n(n-1)}{2} \in \mathbb{Z}$$

for every positive integer n .

Thus, the formula always gives a valid integer number of communication links.

Proofs by Cases

Question 1

A software program accepts any integer input n and computes:

$$f(n) = |n|$$

Show that:

$$|n| \geq 0$$

for every integer n by considering the possible cases for n .

Question 1 – Answer

The two possible cases are:

- $n \geq 0$
- $n < 0$

We examine the absolute value separately in each case.

Question 1 – Answer

Case 1: $n \geq 0$

By definition:

$$|n| = n$$

Since $n \geq 0$:

$$|n| \geq 0$$

Question 1 – Answer

Case 2: $n < 0$

By definition:

$$|n| = -n$$

Since $n < 0$:

$$-n > 0$$

Therefore:

$$|n| \geq 0$$

Hence, in both cases:

$$\boxed{|n| \geq 0}$$

Question 2

A hashing algorithm produces a hash value using:

$$h(n) = n^2$$

For an integer input n , determine whether the hash value has the same parity as n .
Prove your conclusion by considering the cases where n is even and where n is odd.

Question 2 – Answer

There are two possible cases.

- n is even
- n is odd

We examine the parity of n^2 in each case.

Question 2 – Answer

Case 1: n is even

Let:

$$n = 2k$$

Then:

$$n^2 = (2k)^2$$

$$n^2 = 4k^2 = 2(2k^2)$$

Therefore, n^2 is even.

Question 2 – Answer

Case 2: n is odd

Let:

$$n = 2k + 1$$

Then:

$$n^2 = (2k + 1)^2$$

$$n^2 = 4k^2 + 4k + 1$$

$$n^2 = 2(2k^2 + 2k) + 1$$

Therefore, n^2 is odd.

Hence, n^2 has the same parity as n .

Question 2 – Answer

Therefore:

n^2 has the same parity as n

for every integer n .

Question 3

A scheduling system assigns an integer priority value n to a task. The system calculates:

$$g(n) = n(n + 1)$$

Show that the calculated value is always even by considering the two possible parity cases for n .

Question 3 – Answer

There are two cases:

- n is even
- n is odd

We prove that $n(n + 1)$ is even in both cases.

Question 3 – Answer

Case 1: n is even

Let:

$$n = 2k$$

Then:

$$n(n + 1) = 2k(2k + 1)$$

Therefore:

$$n(n + 1) = 2[k(2k + 1)]$$

Hence, the result is even.

Question 3 – Answer

Case 2: n is odd

Let:

$$n = 2k + 1$$

Then:

$$n(n + 1) = (2k + 1)(2k + 2)$$

Therefore:

$$n(n + 1) = 2[(2k + 1)(k + 1)]$$

Hence, the result is also even.

Question 3 – Answer

Since both possible cases produce an even result:

$n(n + 1)$ is always even

Question 4

A computer program compares two integers a and b and returns the larger value:

$$\max(a, b)$$

Prove that:

$$\max(a, b) \geq a$$

for all integers a and b by considering the possible relationships between a and b .

Question 4 – Answer

There are two cases:

- $a \geq b$
- $a < b$

We consider the value returned by the program in each case.

Question 4 – Answer

Case 1: $a \geq b$

In this case:

$$\max(a, b) = a$$

Therefore:

$$\max(a, b) \geq a$$

because equality holds.

Question 4 – Answer

Case 2: $a < b$

In this case:

$$\max(a, b) = b$$

Since:

$$a < b$$

we have:

$$b > a$$

Therefore:

$$\max(a, b) > a$$

Question 4 – Answer

Both possible cases satisfy the required inequality.
Hence:

$$\max(a, b) \geq a$$

for all integers a and b .

Question 5

A data-processing algorithm assigns a cost to an integer input n using:

$$C(n) = \begin{cases} n + 2, & n \geq 0 \\ -n + 2, & n < 0 \end{cases}$$

Show that:

$$C(n) \geq 2$$

for every integer n by proving the result separately for the two possible cases.

Question 5 – Answer

The definition of $C(n)$ naturally gives two cases:

- $n \geq 0$
- $n < 0$

We prove the required inequality in each case.

Question 5 – Answer

Case 1: $n \geq 0$

The function is:

$$C(n) = n + 2$$

Since:

$$n \geq 0$$

we obtain:

$$n + 2 \geq 2$$

Therefore:

$$C(n) \geq 2$$

Question 5 – Answer

Case 2: $n < 0$

The function is:

$$C(n) = -n + 2$$

Since:

$$n < 0$$

we have:

$$-n > 0$$

Therefore:

$$-n + 2 > 2$$

and hence:

$$C(n) \geq 2$$

Question 5 – Answer

Both possible cases satisfy the required condition.
Therefore:

$$C(n) \geq 2$$

for every integer n .

The result follows by examining all cases defined by the function.

Proof of an Implication

Question 1

A software system claims that if a user's access score is divisible by 4, then the score is even.

Prove the implication directly for every integer access score.

Question 1 – Answer

Let n be an integer such that:

$$4 \mid n$$

Then, by the definition of divisibility, there exists an integer k such that:

$$n = 4k$$

Question 1 – Answer

Rewrite n as:

$$n = 2(2k)$$

Since $2k$ is an integer, n has the form $2m$ for an integer m .

Therefore, n is even.

Hence:

$$4 \mid n \Rightarrow 2 \mid n$$

Question 2

A cloud security system follows the rule:

If a server passes authentication, then it is allowed to access the network.

A particular server has passed authentication.

Question: Using Modus Ponens, determine whether the server is allowed to access the network.

Question 2 – Answer

Let:

P = The server passes authentication

and

Q = The server is allowed to access the network

The security rule is:

$$P \rightarrow Q$$

The given information states:

$$P$$

Question 2 – Answer

We therefore have:

$$P \rightarrow Q$$

and

$$P$$

Modus Ponens states:

$$P \rightarrow Q, \quad P \quad \therefore \quad Q$$

Therefore:

$$\boxed{Q}$$

The server is allowed to access the network.

Question 3

A machine-learning deployment system follows the rule:

If a model is deployed, then it has passed all required tests.

An administrator verifies that the model has **not** passed all the required tests.

Question: Using Modus Tollens, determine whether the model can be considered deployed under this rule.

Question 3 – Answer

Let:

P = The model is deployed

and

Q = The model has passed all required tests

The rule is:

$$P \rightarrow Q$$

The given information is:

$$\neg Q$$

Question 3 – Answer

Modus Tollens states:

$$P \rightarrow Q, \quad \neg Q \quad \therefore \quad \neg P$$

Since the model has not passed all required tests,

$$\neg Q$$

Therefore:

$$\boxed{\neg P}$$

The model cannot be considered deployed under the given rule.

Question 4

A program produces the value:

$$f(n) = n^2 + n$$

for an integer input n .

Prove that $f(n)$ is even for every integer n .

Question 4 – Answer

We need to prove:

$$n \in \mathbb{Z} \Rightarrow n^2 + n \text{ is even}$$

Factor the expression:

$$n^2 + n = n(n + 1)$$

Question 4 – Answer

Among any two consecutive integers, one must be even.
Therefore, either:

$$2 \mid n$$

or:

$$2 \mid (n + 1)$$

Thus, the product

$$n(n + 1)$$

must be even.

Question 4 – Answer

Hence:

$$n^2 + n \text{ is even}$$

for every integer n .

Question 5

A computing system represents a positive integer n using its binary representation. Prove that if n is odd, then its binary representation must end in 1.

Question 5 – Answer

Every positive integer can be written as:

$$n = 2q + r$$

where:

$$r \in \{0, 1\}$$

The remainder r determines the last binary digit.

Question 5 – Answer

If n is odd, then division by 2 gives remainder 1.

Therefore:

$$n = 2q + 1$$

Hence:

$$r = 1$$

Question 5 – Answer

In binary representation, the remainder obtained after division by 2 corresponds to the least significant bit.

Therefore:

An odd integer has a binary representation ending in 1

Proof by Contradiction

Question 1

A scheduling algorithm claims that there is no smallest positive integer. Determine whether this claim is correct. Use proof by contradiction to justify your conclusion.

Question 1 – Answer

The claim is false because 1 is the smallest positive integer.

Assume, for contradiction, that there is no smallest positive integer.

Then there must be some positive integer n that is smaller than every positive integer.

Question 1 – Answer

Since n is positive:

$$n \geq 1$$

But 1 is also a positive integer.

If $n > 1$, then:

$$1 < n$$

which contradicts the assumption that n is smaller than every positive integer.

Question 1 – Answer

Therefore, the assumption leads to a contradiction.

Hence:

1 is the smallest positive integer

and the original claim is false.

Question 2

A database system uses integer identifiers. A developer claims that there can be two consecutive integers that are both even.

Prove that this is impossible using contradiction.

Question 2 – Answer

Assume, for contradiction, that two consecutive integers n and $n + 1$ are both even. Then there exist integers a and b such that:

$$n = 2a$$

and:

$$n + 1 = 2b$$

Question 2 – Answer

Subtract the two equations:

$$(n + 1) - n = 2b - 2a$$

Therefore:

$$1 = 2(b - a)$$

The right-hand side is even, while the left-hand side is odd.
This is a contradiction.

Question 2 – Answer

Therefore, two consecutive integers cannot both be even.

Hence:

No two consecutive integers are both even.

Question 3

A software system requires every integer value in a particular set to be both even and odd.

Show that such an integer cannot exist.

Question 3 – Answer

Assume, for contradiction, that an integer n is both even and odd.
Since n is even:

$$n = 2a$$

for some integer a .

Question 3 – Answer

Since n is also odd:

$$n = 2b + 1$$

for some integer b .

Therefore:

$$2a = 2b + 1$$

which gives:

$$2(a - b) = 1$$

Question 3 – Answer

The left-hand side is even, but the right-hand side is odd.

This is impossible.

Therefore:

No integer is both even and odd.

Question 4

A network engineer claims that the sum of two odd integers can be odd. Determine whether the claim is possible. Prove your conclusion by contradiction.

Question 4 – Answer

Assume, for contradiction, that two odd integers a and b have an odd sum.
Since both are odd:

$$a = 2m + 1$$

and:

$$b = 2n + 1$$

Question 4 – Answer

Their sum is:

$$a + b = 2m + 1 + 2n + 1$$

Therefore:

$$a + b = 2(m + n + 1)$$

Thus, $a + b$ is even.

This contradicts the assumption that the sum is odd.

Question 4 – Answer

Hence:

The sum of two odd integers is always even.

Question 5

A computing algorithm claims that the square root of 2 can be represented exactly as a rational number.

Prove that this is impossible using contradiction.

Question 5 – Answer

Assume, for contradiction, that:

$$\sqrt{2} = \frac{p}{q}$$

where p and q are integers with no common factor.

Squaring both sides:

$$2 = \frac{p^2}{q^2}$$

Question 5 – Answer

Therefore:

$$p^2 = 2q^2$$

Hence p^2 is even.

Therefore p must be even.

Let:

$$p = 2k$$

Question 5 – Answer

Substitute into:

$$p^2 = 2q^2$$

giving:

$$(2k)^2 = 2q^2$$

$$4k^2 = 2q^2$$

Therefore:

$$q^2 = 2k^2$$

So q is also even.

Question 5 – Answer

Thus both p and q are even.

This contradicts the assumption that p and q have no common factor.

Therefore:

$\sqrt{2}$ is irrational

Proving IFF

Question 1

A software system claims that an integer n is even if and only if n^2 is even.
Prove the claim.

Question 1 – Answer

To prove:

$$n \text{ is even} \iff n^2 \text{ is even}$$

we must prove both directions.

First:

$$n \text{ even} \Rightarrow n^2 \text{ even}$$

Question 1 – Answer

Let:

$$n = 2k$$

Then:

$$n^2 = (2k)^2 = 4k^2$$

$$n^2 = 2(2k^2)$$

Therefore, n^2 is even.

Question 1 – Answer

For the reverse direction, assume:

$$n^2 \text{ is even}$$

Suppose n were odd. Then:

$$n = 2k + 1$$

and:

$$n^2 = 4k^2 + 4k + 1$$

which is odd.

This is a contradiction.

Therefore, n is even.

Question 1 – Answer

Both directions have been proved.

Hence:

$$n \text{ is even} \iff n^2 \text{ is even}$$

Question 2

A database stores an integer value n . A validation rule states that n is divisible by 6 if and only if it is divisible by both 2 and 3.
Prove the rule.

Question 2 – Answer

We prove both directions.

First:

$$6 \mid n \Rightarrow (2 \mid n \wedge 3 \mid n)$$

If:

$$n = 6k$$

then:

$$n = 2(3k) = 3(2k)$$

Therefore, $2 \mid n$ and $3 \mid n$.

Question 2 – Answer

For the reverse direction, assume:

$$2 \mid n$$

and:

$$3 \mid n$$

Since 2 and 3 are relatively prime, their product divides n .

Therefore:

$$6 \mid n$$

Question 2 – Answer

Thus both directions hold:

$$6 \mid n \iff (2 \mid n \wedge 3 \mid n)$$

Question 3

A programming system uses an integer n . Prove that n is odd if and only if $n + 1$ is even.

Question 3 – Answer

First direction:

$$n \text{ odd} \Rightarrow n + 1 \text{ even}$$

Let:

$$n = 2k + 1$$

Then:

$$n + 1 = 2k + 2 = 2(k + 1)$$

Therefore, $n + 1$ is even.

Question 3 – Answer

Reverse direction:

$$n + 1 \text{ even} \Rightarrow n \text{ odd}$$

Let:

$$n + 1 = 2k$$

Then:

$$n = 2k - 1$$

$$n = 2(k - 1) + 1$$

Therefore, n is odd.

Question 3 – Answer

Both directions hold.

Hence:

$$n \text{ is odd} \iff n + 1 \text{ is even}$$

Question 4

A network system uses two integer parameters a and b .
Prove that:

$$a + b \text{ is even}$$

if and only if a and b have the same parity.

Question 4 – Answer

First assume a and b have the same parity.

If both are even:

$$a = 2m, \quad b = 2n$$

Then:

$$a + b = 2(m + n)$$

Question 4 – Answer

If both are odd:

$$a = 2m + 1, \quad b = 2n + 1$$

Then:

$$a + b = 2(m + n + 1)$$

Thus, in either case, $a + b$ is even.

Question 4 – Answer

Now assume:

$a + b$ is even

If a and b had different parity, one would be even and the other odd.

Their sum would then be odd.

This contradicts the assumption.

Therefore, a and b have the same parity.

Hence:

$$a + b \text{ is even} \iff a \text{ and } b \text{ have the same parity}$$

Question 5

A data structure contains an integer number of elements n .
Show that $n(n - 1)$ is even if and only if at least one of n and $n - 1$ is even.

Question 5 – Answer

First, suppose at least one of n and $n - 1$ is even.

Since:

$$n(n - 1)$$

is a product containing an even factor, the product is even.

Question 5 – Answer

For the reverse direction, suppose:

$$n(n - 1) \text{ is even}$$

Two consecutive integers cannot both be odd.

Therefore, among n and $n - 1$, at least one must be even.

Thus:

$$n(n - 1) \text{ is even} \iff n \text{ or } n - 1 \text{ is even}$$

Sets

Question 1

A university maintains two sets of students:

$$A = \{\text{students enrolled in AI}\}$$

$$B = \{\text{students enrolled in Data Science}\}$$

A student belongs to both programs if they are present in both sets.

Question: Express the set of students enrolled in at least one of the two programs using set notation.

Question 1

Using the given sets:

$$A = \{\text{AI students}\}$$

$$B = \{\text{Data Science students}\}$$

The students enrolled in at least one program belong to the union of the two sets.

Find:

$$A \cup B$$

Answer 1

The union of two sets contains elements belonging to either set or to both sets.
Therefore,

$$A \cup B = \{x \mid x \in A \text{ or } x \in B\}$$

Hence,

$$\boxed{A \cup B}$$

represents all students enrolled in AI, Data Science, or both.

Question 2

A software company maintains two sets:

$$A = \{\text{employees trained in Python}\}$$

$$B = \{\text{employees trained in Machine Learning}\}$$

Only employees trained in both areas are assigned to a particular AI project.

Question: Which set operation identifies the eligible employees?

Answer 2

The required employees must belong to both sets simultaneously.
Therefore, the required set is the intersection:

$$A \cap B$$

where

$$A \cap B = \{x \mid x \in A \text{ and } x \in B\}$$

Hence,

$$\boxed{A \cap B}$$

represents employees trained in both Python and Machine Learning.

Question 3

A database contains records represented by the universal set

$$U = \{1, 2, 3, 4, 5, 6, 7, 8\}.$$

The records containing missing values are

$$A = \{2, 4, 6, 8\}.$$

Question: Determine the records that do not contain missing values.

Answer 3

The records without missing values are the elements of the universal set that are not in A .

Therefore, we use the complement:

$$A^c = U - A$$

Substituting the values,

$$A^c = \{1, 3, 5, 7\}$$

Hence,

$$A^c = \{1, 3, 5, 7\}$$

Question 4

A machine-learning dataset contains 100 records.

Let

$$A = \{\text{records containing missing values}\}$$

and

$$B = \{\text{records containing duplicate values}\}.$$

Suppose

$$|A| = 30, \quad |B| = 20, \quad |A \cap B| = 8.$$

Question: Determine the number of records requiring at least one type of preprocessing.

Answer 4

Records requiring at least one type of preprocessing belong to

$$A \cup B$$

Using the inclusion-exclusion principle,

$$|A \cup B| = |A| + |B| - |A \cap B|$$

Therefore,

$$\begin{aligned} |A \cup B| &= 30 + 20 - 8 \\ &= 42 \end{aligned}$$

Hence,

$$\boxed{42}$$

records require at least one type of preprocessing.

Question 5

A cybersecurity system monitors two types of events.

Let

$$A = \{\text{failed login events}\}$$

and

$$B = \{\text{suspicious network events}\}.$$

An administrator wants to investigate events that are either failed logins or suspicious network events, but not both.

Question: Express this set using set operations.

Answer 5

Events belonging to A but not B are

$$A - B$$

Events belonging to B but not A are

$$B - A$$

Therefore, events belonging to exactly one of the two sets are

$$(A - B) \cup (B - A)$$

This is the symmetric difference:

$$\boxed{A \triangle B}$$

Hence, $A \triangle B$ represents events that occur in exactly one of the two categories.

Proving Set Equations

Question 1

A data-processing system classifies records using two sets A and B .

A developer claims that processing records belonging to both sets is equivalent to first finding all records in A and then removing those outside B .

Question: Prove the set equation

$$A \cap B = A - (A - B).$$

Answer 1

To prove two sets are equal, we show that an arbitrary element x belongs to the left-hand side exactly when it belongs to the right-hand side.

Let

$$x \in A \cap B.$$

Then

$$x \in A \quad \text{and} \quad x \in B.$$

Since $x \in A$ and $x \notin A - B$,

$$x \in A - (A - B).$$

Therefore,

$$A \cap B \subseteq A - (A - B).$$

Answer 1

Now assume

$$x \in A - (A - B).$$

Then

$$x \in A$$

and

$$x \notin A - B.$$

Since $x \in A$, the fact that $x \notin A - B$ means that $x \in B$.

Therefore,

$$x \in A \cap B.$$

Thus,

$$A - (A - B) \subseteq A \cap B.$$

Hence,

$$A \cap B = A - (A - B).$$

Question 2

In a software system, A represents users with administrator privileges and B represents users who are currently active.

A security rule claims:

The set of active administrators is the same as the active users who are administrators.

Question: Prove

$$A \cap B = B \cap A.$$

Answer 2

Let

$$x \in A \cap B.$$

Then

$$x \in A \quad \text{and} \quad x \in B.$$

Since the order of the conditions does not matter,

$$x \in B \quad \text{and} \quad x \in A.$$

Therefore,

$$x \in B \cap A.$$

Hence,

$$A \cap B \subseteq B \cap A.$$

Answer 2

Similarly, let

$$x \in B \cap A.$$

Then

$$x \in B \quad \text{and} \quad x \in A.$$

Therefore,

$$x \in A \cap B.$$

Thus,

$$B \cap A \subseteq A \cap B.$$

Since both inclusions hold,

$$\boxed{A \cap B = B \cap A}.$$

Question 3

A machine-learning system uses set A for records selected by model 1 and set B for records selected by model 2.

A developer claims that selecting records from either model and then selecting those common to both produces the same result as directly selecting the common records.

Question: Prove

$$A \cap (A \cup B) = A.$$

Answer 3

Let

$$x \in A \cap (A \cup B).$$

Then

$$x \in A$$

and

$$x \in A \cup B.$$

The first condition already establishes that

$$x \in A.$$

Therefore,

$$A \cap (A \cup B) \subseteq A.$$

Answer 3

Now let

$$x \in A.$$

Since $x \in A$, it follows that

$$x \in A \cup B.$$

Therefore,

$$x \in A \cap (A \cup B).$$

Hence,

$$A \subseteq A \cap (A \cup B).$$

Both inclusions hold, so

$$\boxed{A \cap (A \cup B) = A}.$$

Question 4

A database administrator removes records belonging to set B from a collection A . The administrator then claims that removing B from A is equivalent to keeping the records in A that are not in B .

Question: Prove

$$A - B = A \cap B^c.$$

Answer 4

Let

$$x \in A - B.$$

By the definition of set difference,

$$x \in A \quad \text{and} \quad x \notin B.$$

Since $x \notin B$,

$$x \in B^c.$$

Therefore,

$$x \in A \cap B^c.$$

Thus,

$$A - B \subseteq A \cap B^c.$$

Answer 4

Now assume

$$x \in A \cap B^c.$$

Then

$$x \in A \quad \text{and} \quad x \in B^c.$$

Therefore,

$$x \notin B.$$

Hence,

$$x \in A - B.$$

Thus,

$$A \cap B^c \subseteq A - B.$$

Therefore,

$$\boxed{A - B = A \cap B^c}.$$

Question 5

A cybersecurity system uses A for normal network traffic and B for encrypted traffic. A security analyst claims that the complement of traffic that is either normal or encrypted is exactly the traffic that is neither normal nor encrypted.

Question: Prove

$$(A \cup B)^c = A^c \cap B^c.$$

Answer 5

Let

$$x \in (A \cup B)^c.$$

Then

$$x \notin A \cup B.$$

Therefore, x is neither in A nor in B :

$$x \notin A \quad \text{and} \quad x \notin B.$$

Hence,

$$x \in A^c \quad \text{and} \quad x \in B^c.$$

Therefore,

$$x \in A^c \cap B^c.$$

Answer 5

Conversely, suppose

$$x \in A^c \cap B^c.$$

Then

$$x \notin A \quad \text{and} \quad x \notin B.$$

Therefore,

$$x \notin A \cup B.$$

Hence,

$$x \in (A \cup B)^c.$$

Thus,

$$\boxed{(A \cup B)^c = A^c \cap B^c}.$$

Russell's Paradox

Question 1

Consider the collection

$$R = \{x \mid x \notin x\}.$$

In other words, R is intended to contain exactly those sets that do not contain themselves.

Question: Determine whether

$$R \in R$$

can consistently be true or false.

Answer 1

Assume first that

$$R \in R.$$

By the definition of R , every element of R must satisfy

$$x \notin x.$$

Therefore,

$$R \notin R.$$

This contradicts the assumption

$$R \in R.$$

Answer 1

Now assume

$$R \notin R.$$

Since R satisfies the defining condition

$$x \notin x,$$

it should therefore belong to R .

Hence,

$$R \in R.$$

This again produces a contradiction.

Therefore, neither assumption gives a consistent result.

The definition of R leads to a contradiction.

Question 2

A hypothetical programming language allows a collection to contain any object, including itself.

Define

$$S = \{x \mid x \text{ is a collection that does not contain itself}\}.$$

Question: Explain why allowing unrestricted collection formation creates a logical problem when considering whether S contains itself.

Answer 2

Suppose

$$S \in S.$$

By the definition of S , an element can belong to S only if it does not contain itself.
Therefore,

$$S \notin S.$$

This contradicts the assumption.

Conversely, suppose

$$S \notin S.$$

Then S satisfies the defining condition for membership in S , so

$$S \in S.$$

Thus, both possibilities lead to contradiction.

Answer 2

The problem arises because the language permits a collection to be defined using a condition that refers to its own membership.

The resulting condition is self-referential:

$$S \in S \iff S \notin S.$$

Therefore, unrestricted collection formation cannot provide a consistent set corresponding to this definition.

Unrestricted self-referential set formation is problematic.

Question 3

A database system attempts to create a set containing all database tables that do not contain themselves as a table.

Let

$$T = \{x \mid x \notin x\}.$$

The database designer claims that T can simply be treated as an ordinary set.

Question: Use Russell's paradox to show why this claim is problematic.

Answer 3

Consider whether

$$T \in T.$$

If

$$T \in T,$$

then by the definition of T ,

$$T \notin T.$$

Contradiction.

If instead

$$T \notin T,$$

then T satisfies the condition required for membership in T .

Therefore,

$$T \in T.$$

Again, contradiction.

Answer 3

Therefore, the database designer cannot consistently treat T as an ordinary set satisfying the given definition.

The contradiction comes from allowing the collection to refer to its own membership.

$$T \in T \iff T \notin T$$

Thus, the proposed definition is not a valid unrestricted set construction.

Question 4

Consider a software repository containing collections of objects.
Suppose a rule attempts to create

$$R = \{\text{all collections that do not contain themselves}\}.$$

Question: Is the contradiction caused by the particular objects stored in the collections, or by the definition of R itself? Justify your answer using Russell's paradox.

Answer 4

The contradiction does not depend on the particular objects stored in the collections. It arises from the definition

$$R = \{x \mid x \notin x\}.$$

Membership of R is determined by whether an object contains itself. Therefore, the definition directly creates the condition

$$R \in R \iff R \notin R.$$

Answer 4

Thus, changing the objects contained in the collections does not remove the fundamental contradiction.

The issue is the unrestricted self-reference in the definition.

The contradiction is caused by the definition of R .

Question 5

A programmer proposes the following rule for a collection type:

Create a collection containing exactly those collections that do not contain themselves.

Question: Explain why a type system that permits this construction without restrictions would be logically inconsistent.

Answer 5

Let the proposed collection be

$$R = \{x \mid x \notin x\}.$$

There are two possibilities.

If

$$R \in R,$$

then the definition requires

$$R \notin R.$$

If

$$R \notin R,$$

then R satisfies the condition for membership, giving

$$R \in R.$$

Answer 5

Thus,

$$R \in R \iff R \notin R.$$

Both possible membership assignments lead to contradiction.

Therefore, a logical system or type system cannot permit unrestricted construction of such a collection while remaining consistent.

The self-referential construction must be restricted.

Well-Ordering Principle

Question 1

A software system assigns positive integer IDs to tasks.
Suppose a collection of pending tasks is represented by a non-empty subset

$$S \subseteq \mathbb{Z}^+.$$

Question: What property guarantees that there is a task with the smallest ID?

Answer 1

The required property is the **Well-Ordering Principle**.

It states that every non-empty subset of the positive integers contains a least element.
Since

$$S \subseteq \mathbb{Z}^+$$

and S is non-empty,

$$\boxed{\exists m \in S \text{ such that } m \leq x \text{ for every } x \in S.}$$

Therefore, the pending tasks have a smallest ID.

Question 2

A distributed system records retry attempts using positive integer numbers. Let S be the set of retry numbers for which a particular event succeeds. Suppose at least one successful retry exists.

Question: Explain why there must be a first successful retry.

Answer 2

Since at least one successful retry exists,

$$S \neq \emptyset.$$

Also,

$$S \subseteq \mathbb{Z}^+.$$

By the Well-Ordering Principle, every non-empty subset of positive integers has a least element.

Therefore, there exists

$$m \in S$$

such that

$$m \leq x \quad \forall x \in S.$$

Hence, m is the first successful retry.

Question 3

Consider the set

$$S = \{n \in \mathbb{Z}^+ \mid n^2 > 20\}.$$

A program searches for the smallest positive integer satisfying the condition.

Question: Use the Well-Ordering Principle to justify why a smallest valid value exists.

Answer 3

First, observe that S is non-empty because

$$5^2 = 25 > 20.$$

Therefore,

$$5 \in S.$$

Also,

$$S \subseteq \mathbb{Z}^+.$$

By the Well-Ordering Principle, S must contain a least element.

Answer 3

We can identify the least element directly.

For positive integers less than 5,

$$1^2 = 1, \quad 2^2 = 4, \quad 3^2 = 9, \quad 4^2 = 16.$$

None is greater than 20.

But

$$5^2 = 25 > 20.$$

Therefore,

$$\boxed{\min(S) = 5}.$$

Question 4

A programmer claims:

Every non-empty subset of the integers has a smallest element.

Question: Is this statement true? If not, provide a counterexample and explain why the Well-Ordering Principle does not apply.

Answer 4

The statement is false for arbitrary subsets of the integers.

Consider

$$S = \{\dots, -3, -2, -1\}.$$

This set is non-empty, but it has no smallest element.

For any

$$n \in S,$$

the integer

$$n - 1$$

also belongs to S and satisfies

$$n - 1 < n.$$

Answer 4

The Well-Ordering Principle applies specifically to non-empty subsets of the positive integers.

It does not state that every non-empty subset of $\mathbb{Z}$ has a least element.

Therefore,

The programmer's claim is false.

Question 5

A search algorithm examines positive integer candidate values and succeeds for at least one candidate.

Let S be the set of successful candidate values.

The algorithm always chooses the smallest successful candidate.

Question: Explain mathematically why this strategy is well-defined.

Answer 5

Since the algorithm succeeds for at least one candidate,

$$S \neq \emptyset.$$

Every candidate is a positive integer, so

$$S \subseteq \mathbb{Z}^+.$$

By the Well-Ordering Principle, S contains a least element.
Therefore,

$$\exists m \in S$$

such that

$$m \leq x \quad \forall x \in S.$$

Answer 5

Hence, the instruction
choose the smallest successful candidate
is mathematically well-defined.
The algorithm can select

$$m = \min(S).$$

The existence of this minimum follows from the Well-Ordering Principle.

Induction

Question 1

A program processes n records. The number of elementary processing steps is modeled by

$$T(n) = 1 + 2 + \cdots + n.$$

A developer claims that

$$T(n) = \frac{n(n+1)}{2}$$

for every positive integer n .

Question: Prove the claim using mathematical induction.

Answer 1

Define the statement

$$P(n) : 1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

Base case: $n = 1$

$$1 = \frac{1(1+1)}{2} = 1.$$

Therefore, $P(1)$ is true.

Answer 1

Inductive hypothesis:

Assume $P(k)$ is true:

$$1 + 2 + \cdots + k = \frac{k(k+1)}{2}.$$

We must prove $P(k+1)$:

$$1 + 2 + \cdots + k + (k+1) = \frac{(k+1)(k+2)}{2}.$$

Answer 1

Using the inductive hypothesis,

$$\begin{aligned}1 + 2 + \cdots + k + (k + 1) \\&= \frac{k(k + 1)}{2} + (k + 1) \\&= \frac{k(k + 1) + 2(k + 1)}{2} \\&= \frac{(k + 1)(k + 2)}{2}.\end{aligned}$$

Therefore, $P(k + 1)$ is true. Hence,

$$1 + 2 + \cdots + n = \frac{n(n + 1)}{2}$$

for every positive integer n .

Question 2

A data structure stores a sequence of nodes. Suppose the number of operations required to process the first n nodes is

$$T(n) = 1 + 3 + 5 + \cdots + (2n - 1).$$

Question: Prove using mathematical induction that

$$T(n) = n^2$$

for every positive integer n .

Answer 2

Define

$$P(n) : 1 + 3 + 5 + \cdots + (2n - 1) = n^2.$$

Base case: $n = 1$

$$1 = 1^2.$$

Therefore, $P(1)$ is true.

Answer 2

Inductive hypothesis:

Assume

$$1 + 3 + 5 + \cdots + (2k - 1) = k^2.$$

We must prove

$$1 + 3 + 5 + \cdots + (2k - 1) + (2k + 1) = (k + 1)^2.$$

Using the hypothesis,

$$k^2 + (2k + 1) = k^2 + 2k + 1 = (k + 1)^2.$$

Answer 2

Therefore,

$$1 + 3 + 5 + \cdots + (2k + 1) = (k + 1)^2.$$

Thus, $P(k + 1)$ is true.

Hence, by mathematical induction,

$$1 + 3 + 5 + \cdots + (2n - 1) = n^2$$

for every positive integer n .

Question 3

A data structure contains a binary tree with n levels. Suppose the maximum number of nodes at level i is

$$2^{i-1}.$$

Question: Prove that the maximum total number of nodes in a binary tree with n levels is

$$2^n - 1.$$

Answer 3

Let

$$P(n) : 1 + 2 + 2^2 + \cdots + 2^{n-1} = 2^n - 1.$$

Base case: $n = 1$

$$1 = 2^1 - 1 = 1.$$

Therefore, $P(1)$ is true.

Answer 3

Assume

$$1 + 2 + \cdots + 2^{k-1} = 2^k - 1.$$

For $k + 1$ levels,

$$1 + 2 + \cdots + 2^{k-1} + 2^k.$$

Using the hypothesis,

$$= (2^k - 1) + 2^k$$

$$= 2^{k+1} - 1.$$

Therefore, $P(k + 1)$ is true.

Answer 3

Hence, by mathematical induction,

$$1 + 2 + 2^2 + \cdots + 2^{n-1} = 2^n - 1$$

Therefore, the maximum number of nodes in a binary tree with n levels is

$$2^n - 1.$$

Question 4

A university maintains three sets of students:

A = students enrolled in Artificial Intelligence,

B = students enrolled in Data Science,

and

C = students enrolled in Cybersecurity.

To avoid counting the same student multiple times, the university uses the inclusion–exclusion principle.

Question: Prove the three-set inclusion–exclusion formula using mathematical induction as a step toward the general principle.

Answer 4

For three sets, the inclusion–exclusion formula is

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|.$$

For the general principle, define

$$P(n) : \left| \bigcup_{i=1}^n A_i \right| = \sum_i |A_i| - \sum_{i < j} |A_i \cap A_j| + \sum_{i < j < k} |A_i \cap A_j \cap A_k| - \dots .$$

Answer 4

The formula follows by counting each element according to the number of sets containing it.

Suppose an element belongs to exactly r of the sets.

It is counted:

$$\binom{r}{1}$$

times in the single-set terms,

$$\binom{r}{2}$$

times in the pairwise terms, and so on.

Therefore, its total contribution is

$$\binom{r}{1} - \binom{r}{2} + \binom{r}{3} - \cdots + (-1)^{r+1} \binom{r}{r}.$$

Answer 4

Using the binomial identity

$$(1 - 1)^r = \binom{r}{0} - \binom{r}{1} + \binom{r}{2} - \cdots + (-1)^r \binom{r}{r},$$

we obtain

$$\binom{r}{1} - \binom{r}{2} + \cdots + (-1)^{r+1} \binom{r}{r} = 1.$$

Thus, every element belonging to the union is counted exactly once.

Hence,

$$\left| \bigcup_{i=1}^n A_i \right| = \sum_{k=1}^n (-1)^{k+1} \sum_{1 \leq i_1 < \cdots < i_k \leq n} |A_{i_1} \cap \cdots \cap A_{i_k}|$$

Question 5

Consider a recursive algorithm defined by

$$F(1) = 1$$

and

$$F(n) = F(n - 1) + 2 \quad (n > 1).$$

A programmer claims that

$$F(n) = 2n - 1.$$

Question: Prove the claim using mathematical induction.

Answer 5

Let

$$P(n) : F(n) = 2n - 1.$$

Base case:

$$F(1) = 1$$

and

$$2(1) - 1 = 1.$$

Therefore, $P(1)$ is true.

Answer 5

Assume

$$F(k) = 2k - 1.$$

Using the recursive definition,

$$F(k + 1) = F(k) + 2.$$

By the inductive hypothesis,

$$F(k + 1) = (2k - 1) + 2 = 2k + 1.$$

Therefore,

$$F(k + 1) = 2(k + 1) - 1.$$

Answer 5

Thus, $P(k + 1)$ is true.

Since the base case is true and the inductive step is valid,

$$F(n) = 2n - 1$$

for every positive integer n .

Invariants

Question 1

A program starts with an integer counter $x = 0$.

At every step, the program either increases x by 2 or decreases x by 2.

Question: Identify an invariant of the counter and explain why it remains true after every operation.

Answer 1

Initially,

$$x = 0,$$

which is even.

Suppose x is even before an operation.

The program changes x to either

$$x + 2$$

or

$$x - 2.$$

Both 2 and -2 are even.

Answer 1

Therefore,

$$\text{even} + \text{even} = \text{even}.$$

Hence, after either operation, x remains even.

Thus, the invariant is

x is always even.

Question 2

A token is placed on position 0 of a one-dimensional grid.

At each step, the token moves either 2 positions to the left or 2 positions to the right.

Question: Can the token ever reach position 5? Use an invariant to justify your answer.

Answer 2

The token starts at

$$x = 0,$$

which is even.

Each move changes the position by either

$$+2 \quad \text{or} \quad -2.$$

Therefore, the parity of the position never changes.

The invariant is

x is always even.

Answer 2

Since 5 is odd,

$$5 \not\equiv 0 \pmod{2}.$$

But every reachable position is even.

Therefore, position 5 cannot be reached.

The token can never reach position 5.

Question 3

A distributed system has two counters A and B .
Initially,

$$A + B = 20.$$

At every operation, the system increases A by 1 and decreases B by 1, or decreases A by 1 and increases B by 1.

Question: Determine an invariant of the system.

Answer 3

Initially,

$$A + B = 20.$$

Consider the first type of operation:

$$A \rightarrow A + 1, \quad B \rightarrow B - 1.$$

The new sum is

$$(A + 1) + (B - 1) = A + B.$$

Answer 3

For the second operation,

$$A \rightarrow A - 1, \quad B \rightarrow B + 1.$$

The new sum is

$$(A - 1) + (B + 1) = A + B.$$

Therefore, neither operation changes the sum.

Hence,

$$A + B = 20$$

is an invariant of the system.

Question 4

A program contains two integer variables x and y .
Initially,

$$x + y = 10.$$

The program repeatedly replaces (x, y) by either

$$(x + 1, y - 1)$$

or

$$(x - 1, y + 1).$$

Question: After any number of operations, what can be concluded about $x + y$?

Answer 4

Initially,

$$x + y = 10.$$

For the first operation,

$$(x + 1) + (y - 1) = x + y = 10.$$

For the second operation,

$$(x - 1) + (y + 1) = x + y = 10.$$

Thus, both possible operations preserve the sum.

Answer 4

Therefore, regardless of the number of operations,

$$\boxed{x + y = 10}.$$

The quantity $x + y$ is an invariant of the program.

This allows us to reason about the program without examining every possible sequence of operations.

Question 5

A board contains tokens placed on grid positions.

Initially, the number of black tokens equals the number of white tokens.

An operation removes one black token and one white token simultaneously.

Question: Show that the difference between the number of black and white tokens is an invariant.

Answer 5

Let

B = number of black tokens

and

W = number of white tokens.

Initially,

$$B = W.$$

Therefore,

$$B - W = 0.$$

Answer 5

After one operation,

$$B \rightarrow B - 1$$

and

$$W \rightarrow W - 1.$$

The new difference is

$$(B - 1) - (W - 1) = B - W.$$

Therefore, the difference does not change.

Answer 5

Hence,

$$B - W = 0$$

remains true after every operation.

Therefore, the equality

$$B = W$$

is an invariant of the process.

Strong Induction

Question 1

A recursive algorithm can solve a problem of size n by using a solution to any smaller positive problem size.

Suppose every integer $n \geq 2$ can be reduced to either $n - 1$ or $n - 2$.

Question: Prove using strong induction that every positive integer can be reached from 1 by repeatedly adding 1 or 2.

Answer 1

Let $P(n)$ be the statement:

n can be reached from 1 by repeatedly adding 1 or 2.

Base cases:

1

is already reached.

Also,

$$1 + 1 = 2.$$

Thus, $P(1)$ and $P(2)$ are true.

Answer 1

Strong inductive hypothesis:

Assume that

$$P(1), P(2), \dots, P(k)$$

are all true.

We must prove $P(k + 1)$.

Since $k + 1 \geq 2$, we can obtain it from the smaller integer k by adding 1:

$$k + 1 = k + 1.$$

By the inductive hypothesis, k can be reached from 1.

Answer 1

Therefore, after reaching k , adding 1 reaches $k + 1$.

Hence $P(k + 1)$ is true.

Thus, by strong induction,

every positive integer can be reached from 1.

Question 2

A system represents an integer amount using coins of denominations 1 and 2.

Question: Prove using strong induction that every positive integer amount can be represented using these coins.

Answer 2

Let $P(n)$ denote:

n can be represented using coins of denominations 1 and 2.

Base cases:

$$1 = 1$$

and

$$2 = 2.$$

Therefore, $P(1)$ and $P(2)$ are true.

Answer 2

Assume that

$$P(1), P(2), \dots, P(k)$$

are all true.

Consider $k + 1$.

Since $k \geq 2$,

$$k + 1 = k + 1.$$

By the inductive hypothesis, k can be represented using the available coins.

Adding one coin of denomination 1 gives a representation of $k + 1$.

Answer 2

Therefore, $P(k + 1)$ is true.
Hence, by strong induction,

every positive integer can be represented using coins of denominations 1 and 2.

Question 3

A recursive program solves a problem of size n by first solving problems of sizes smaller than n .

Suppose the program can solve the problem directly for sizes 1 and 2, and for every $n \geq 3$, a solution for size $n - 1$ is sufficient.

Question: Explain why strong induction is suitable for proving that the program works for every positive integer input size.

Answer 3

Let $P(n)$ denote:

The program correctly solves the problem for input size n .

The base cases are

$$P(1) \quad \text{and} \quad P(2).$$

For the inductive step, assume

$$P(1), P(2), \dots, P(k)$$

are all true.

Answer 3

For input size $k + 1$, the program requires a correct solution for a smaller input size, specifically k .

Since

$$P(k)$$

is included in the strong inductive hypothesis, the required smaller problem is known to be solvable.

Therefore,

$$P(k + 1)$$

is true.

Hence, strong induction proves correctness for every positive input size.

Question 4

Consider the Fibonacci sequence:

$$F_1 = 1, \quad F_2 = 1$$

and

$$F_n = F_{n-1} + F_{n-2}.$$

Question: Prove using strong induction that

$$F_n < 2^n$$

for every positive integer n .

Answer 4

Let

$$P(n) : F_n < 2^n.$$

For $n = 1$,

$$F_1 = 1 < 2.$$

For $n = 2$,

$$F_2 = 1 < 4.$$

Thus, the base cases are true.

Answer 4

Assume strongly that

$$F_i < 2^i$$

for every

$$1 \leq i \leq k.$$

We prove the statement for $k + 1$.

Using the recurrence,

$$F_{k+1} = F_k + F_{k-1}.$$

By the inductive hypothesis,

$$F_k < 2^k$$

and

$$F_{k-1} < 2^{k-1}.$$

Answer 4

Therefore,

$$F_{k+1} < 2^k + 2^{k-1}$$

$$< 2^k + 2^k \\ = 2^{k+1}.$$

Hence,

$$F_{k+1} < 2^{k+1}.$$

Therefore,

$$\boxed{F_n < 2^n}$$

for every positive integer n .

Question 5

A file-compression algorithm represents files using blocks of size 2 or 3 units.

Question: Prove using strong induction that every integer $n \geq 2$ can be represented as a sum of 2s and 3s, except $n = 1$.

Answer 5

Let $P(n)$ denote:

n can be expressed as a sum of $2s$ and $3s$.

The initial cases are

$$2 = 2,$$

$$3 = 3,$$

and

$$4 = 2 + 2.$$

Therefore,

$$P(2), P(3), P(4)$$

are true.

Answer 5

Assume strongly that

$$P(2), P(3), \dots, P(k)$$

are all true.

Consider $k + 1$.

For $k + 1 \geq 5$,

$$k + 1 = (k - 1) + 2.$$

Since

$$k - 1 \geq 4,$$

the strong inductive hypothesis guarantees that $k - 1$ can be represented using 2s and 3s.

Answer 5

Adding one more 2 gives a representation of $k + 1$.
Therefore,

$$P(k + 1)$$

is true.

Hence, by strong induction,

every integer $n \geq 2$ can be represented using 2s and 3s.

Structural Induction

Question 1

A programming language defines arithmetic expressions recursively:

- Every variable is an expression.
- If E and F are expressions, then $(E + F)$ is an expression.

Question: Prove by structural induction that every expression contains an equal number of opening and closing parentheses.

Answer 1

Let $P(E)$ be the statement:

Expression E contains the same number of opening and closing parentheses.

Base case:

A variable such as x contains no parentheses.

Therefore,

$$0 = 0.$$

Hence, $P(E)$ is true for every basic variable expression.

Answer 1

Inductive step:

Assume E and F are expressions satisfying P .

Therefore, each has an equal number of opening and closing parentheses.

The new expression is

$$(E + F).$$

This adds one opening parenthesis and one closing parenthesis.

Thus, equality is preserved.

Answer 1

Therefore, the resulting expression also contains equal numbers of opening and closing parentheses.

Hence, by structural induction,

Every valid expression has equally many opening and closing parentheses.

Question 2

Consider a recursively defined set of binary strings:

- The empty string ϵ belongs to the set.
- If s belongs to the set, then $0s1$ also belongs to the set.

Question: Prove by structural induction that every string in the set contains the same number of 0s and 1s.

Answer 2

Let $P(s)$ be:

$$\#0(s) = \#1(s).$$

Base case:

For the empty string,

$$s = \epsilon.$$

It contains no 0s and no 1s.

Therefore,

$$\#0(\epsilon) = \#1(\epsilon) = 0.$$

Answer 2

Inductive step:

Assume

$$\#0(s) = \#1(s).$$

The construction rule creates

$$0s1.$$

This adds exactly one 0 and one 1.

Therefore,

$$\#0(0s1) = \#0(s) + 1$$

and

$$\#1(0s1) = \#1(s) + 1.$$

Answer 2

Since

$$\#0(s) = \#1(s),$$

we obtain

$$\#0(0s1) = \#1(0s1).$$

Hence, by structural induction,

Every string in the set contains equally many 0s and 1s.

Question 3

A simple expression language is defined recursively:

- Every integer constant is an expression.
- If E and F are expressions, then $(E - F)$ is an expression.

Question: Prove by structural induction that every expression contains an equal number of opening and closing parentheses.

Answer 3

Let $P(E)$ denote:

E contains the same number of opening and closing parentheses.

Base case:

An integer constant such as 5 contains no parentheses.

Thus,

$$0 = 0.$$

Therefore, the property holds for every basic expression.

Answer 3

Inductive step:

Assume E and F satisfy the property.

Construct

$$(E - F).$$

The expressions E and F already contain equal numbers of both types of parentheses.

The construction adds exactly one opening and one closing parenthesis.

Therefore, the equality remains true.

Answer 3

Hence, the new expression satisfies the property.

By structural induction,

every expression contains equally many opening and closing parentheses.

Question 4

A programming language defines binary trees recursively:

- A single node is a binary tree.
- If T_1 and T_2 are binary trees, a new tree can be formed with a root and T_1, T_2 as its subtrees.

Question: Prove by structural induction that every binary tree contains one more node than the number of edges.

Answer 4

Let

$$P(T) : \text{ number of nodes} = \text{ number of edges} + 1.$$

Base case:

A single-node tree has

1 node

and

0 edges.

Therefore,

$$1 = 0 + 1.$$

Answer 4

Inductive hypothesis:

Assume T_1 and T_2 satisfy

$$N_1 = E_1 + 1$$

and

$$N_2 = E_2 + 1.$$

Construct a new tree with a root and subtrees T_1, T_2 .

The number of nodes is

$$N = 1 + N_1 + N_2.$$

Answer 4

The number of edges is

$$E = 2 + E_1 + E_2,$$

because the root contributes two edges to the two subtrees.

Using the inductive hypotheses,

$$\begin{aligned} N &= 1 + (E_1 + 1) + (E_2 + 1) \\ &= E_1 + E_2 + 3 \end{aligned}$$

and

$$E + 1 = (E_1 + E_2 + 2) + 1 = E_1 + E_2 + 3.$$

Therefore,

$$N = E + 1.$$

Answer 4

Hence, by structural induction,

Every binary tree has exactly one more node than edges.

Question 5

A recursively defined set of expressions is generated as follows:

- Every variable is an expression.
- If E is an expression, then (E) is an expression.
- If E and F are expressions, then $(E + F)$ is an expression.

Question: Prove by structural induction that every expression contains at least one variable.

Answer 5

Let $P(E)$ denote:

E contains at least one variable.

Base case:

Every variable is an expression.

For example,

x

contains one variable.

Therefore, $P(E)$ is true for every basic expression.

Answer 5

Inductive step 1:

Assume E contains at least one variable.

The construction rule produces

(E) .

Parentheses do not remove the variable from E .

Therefore, (E) still contains at least one variable.

Answer 5

Inductive step 2:

Assume E and F each contain at least one variable.

The construction

$$(E + F)$$

contains all variables from both E and F .

Therefore, it contains at least one variable.

Hence, by structural induction,

Every expression contains at least one variable.

Thank You